



PROJECT-BASED LEARNING

PHASE-I EVALUATION

Intelligent LoadBalancer with Server Failure Handling

Team ID: T213

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026-27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T213
Semester: 3rd
Section: ML1,ML2,ML3
Domain: *Computer Networks*

Team Members

1. *Prakash Chandra Pandey – 2512510008*
2. *Tirth Sharma – 2510370377*
3. *Harsh Rastogi – 2510370310*

Mentor

Mr. Manish Mehta sir

Interactions completed: 3

Problem Statement & Motivation

Problem Statement

When many requests are sent at the same time, a server can become overloaded while other servers are use less Also, if one server fails, requests sent to it may not be processed. This project aims to develop a load balancing system that distributes requests among available servers and handles server failures sending new incoming requests to working servers. The system will also provide a simple GUI to show the request distribution and server status.

Motivation

- ▶ *Uneven requests cause server overload*
- ▶ *Server failure affects request handling.*
- ▶ *Existing system are complex to learn.*

Target Users / Stakeholders

- ▶ *Students and developers*
- ▶ *System administrator*
- ▶ *Easy load monitoring*

Objectives

Project Objectives

1. *Develop a load balancer to distribute incoming requests among multiple servers.*
2. *Implement the Round Robin algorithm for request distribution.*
3. *Use queues to manage pending requests and server workloads and use classes for load balancer*
4. *Test request distribution and server failure handling.*
5. *Demonstrate practical applicability and future scope.*

Key Objective: *Efficiently distribute incoming requests while their is a server failure.*

Proposed Solution

Proposed Approach

1. *Input server and request through GUI.*
2. *Round Robin Request Distribution.*
3. *Server Failure handling*
4. *Storing requests in queues.*
5. *Load balancing through GUI display*

Key Features

- ▶ *Round Robin Request Distribution*
- ▶ *Server failure handling*
- ▶ *Manual Server Rebooting*
- ▶ *Request displays in server*
- ▶ *Interactive GUI display*

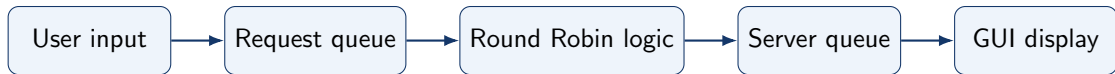
Clearly explain how the proposed solution addresses the identified problem.

Integration of PBL Course Concepts

Course	Concepts Applied	Project Module
Data Structures in C	<i>Queues, Priority Queues, Vectors, Dynamic arrays</i>	<i>Request Queue and server queues</i>
OOPs with C++	<i>Classes, object, encapsulation, inheritance STL.</i>	<i>Server Class, Request Class, Load Balancer Class and Server Management</i>

Requirement: Demonstrate meaningful integration of both subjects in **one** project.

System Architecture / Workflow



Major Components

- ▶ *GUI and input control module*
- ▶ *Request Queue and Server Queues*
- ▶ *Round Robin Load Balancer*
- ▶ *Server Failure and Rebooting*

Data / Control Flow

- ▶ *Incoming requests in queue*
- ▶ *Load balancer checks server status and assigns queued requests using Round Robin.*
- ▶ *GUI-based request distribution and server failure handling*

Technology Stack & Methodology

Technology Stack

- ▶ Programming: *C++*
- ▶ Database: *None(In-Memory/Queue*
- ▶ Tools / Framework: *Qt widgets*
- ▶ IDE: *Visual Studio Code*
- ▶ Version Control: *Git / GitHub*

Development Methodology

1. Planning and scope
2. System and module design
3. OOP , DS and GUI implementation
4. Testing
5. Evaluation

Justify the selection of technologies and methodology.

Initial Progress & Team Contribution

Progress Achieved

- ▶ *Project topic and objectives finalized*
- ▶ *Requirements and user roles identified*
- ▶ *Load balancer and server recovery architecture designed*
- ▶ *Request queue implemented*

Individual Contribution

Member	Role	Contribution
Prakash Chandra Pandey	Lead	GUI integration, overall system testing and planning
Tirth Sharma	Developer	Round Robin scheduling algorithm, backend logic
Harsh Rastogi	Developer/Docs	OOP class design, Documentation, server queues

Roadmap, Expected Outcomes & References

Roadmap

1. Phase-I: Analysis and System Proposal
2. Phase-II: C++ Backend and Qt GUI Development.
3. Phase-III: Integration , Failure Testing and Final Implementation.

Expected Outcomes

- ▶ *Working Load balancer system*
- ▶ *Efficient request distribution*
- ▶ *Server failure handling*
- ▶ *Future server expansion*

References

1. *Load balancing in distributed systems*
2. *Data Structures and Algorithms book by G.S. Baluja*
3. *Qt Documentation — Qt Framework*
4. *QT tutorial*

Thank You
Questions & Suggestions